FUJINET FOR THE INTELLIVISION

JLP + FUJINET

Sharing the Cartridge Window

A companion to the FujiNet Programmer's Guide for Intellivision. How JLP and the FujiNet mailbox share `$8000`–`$9FFF`, and the test cartridge that proves it.

FujiNet is a community project. Intellivision is a registered trademark of its respective owner; this booklet is not affiliated with or endorsed by Mattel.

TABLE OF CONTENTS

CHAPTER 1 One Window, Two Tenants 1

CHAPTER 2 How the Cartridge Decides 3

CHAPTER 3 Inside the Cartridge 5

CHAPTER 4 Writing a FujiNet-Aware JLP Game 7

CHAPTER 5 The Test Cartridge 9

CHAPTER 6 Emulators and Known Issues 11

Two tenants, one window, and a lease both of them can read.

CHAPTER 1

ONE WINDOW, TWO TENANTS

What JLP is, what the FujiNet mailbox is, and why they meet at the same addresses.

JLP is Joe Zbiciak's "Jean-Luc Project" cartridge board. Many homebrew carts implement it, and IntyBASIC's `--jlp` switch targets it. It gives a game 8,000 words of 16-bit RAM at `$8040`–`$9F7F`, hardware multiply and divide, a CRC-16 unit, a random-number register, and save-game flash in rows of 96 words.

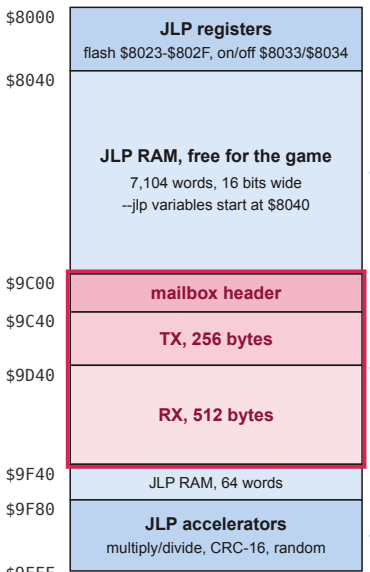
FujiNet reaches the Intellivision as a cartridge with two processors: an RP2040 or RP2350 that answers the console's bus, and an ESP32-S3 running the FujiNet firmware, joined by USB. Your program talks to FujiNet through the **mailbox**: 832 words of cartridge RAM at `$9C00`–`$9F3F` with a header, a 256-byte transmit buffer and a 512-byte receive buffer.

The cartridge emulates JLP as well, so JLP games run on it, and inside the cartridge JLP RAM and the mailbox are one and the same array. A game that wants both simply gives up the mailbox's 832 words of JLP RAM.

RANGE	OWNER	NOTES
`$8000-$803F`	JLP	Registers: flash rows, flash commands, accelerators on/off
`$8040-$9BFF`	JLP	7,104 words of RAM for the game
`$9C00-$9F3F`	FujiNet	The mailbox, when the game opts in; otherwise plain JLP RAM
`$9F40-$9F7F`	JLP	64 more words of RAM
`$9F80-$9FFF`	JLP	Multiply/divide, CRC-16, random numbers

One window, two tenants: $8000-$9FFF

Not to scale. Blue is JLP's, magenta is FujiNet's. Both are the same RAM inside the cartridge.



Keep every variable below $9C00

IntyBASIC only checks against $9F80, so a big DIM can run into the mailbox without a warning. Check _SYSTEM2 in the .sym file.

FujiNet mailbox, $9C00-$9F3F

Exists for a JLP game only when its .cfg says [vars] fujinet = 1. Byte-wide cells: MAGIC F,N SEQ $9C03 ACKSEQ $9C04 LINK $9C0F Touch it only through the mailbox protocol. Without the opt-in this is ordinary JLP RAM.

Never shared

$9F40-$9FFF belongs to JLP even when the mailbox is on. Flash commands may not copy a row into or out of the mailbox.

CHAPTER 2

HOW THE CARTRIDGE DECIDES

The four ways the cart can map $8000–$9FFF, and the one line that chooses between them.

When CONFIG boots a game, the ESP32 pushes the ROM **and its .cfg file** to the cartridge, and the cartridge maps the game from the .cfg. Three `[vars]` keys matter here: `jlp`, `jlpflash` and `fujinet`.

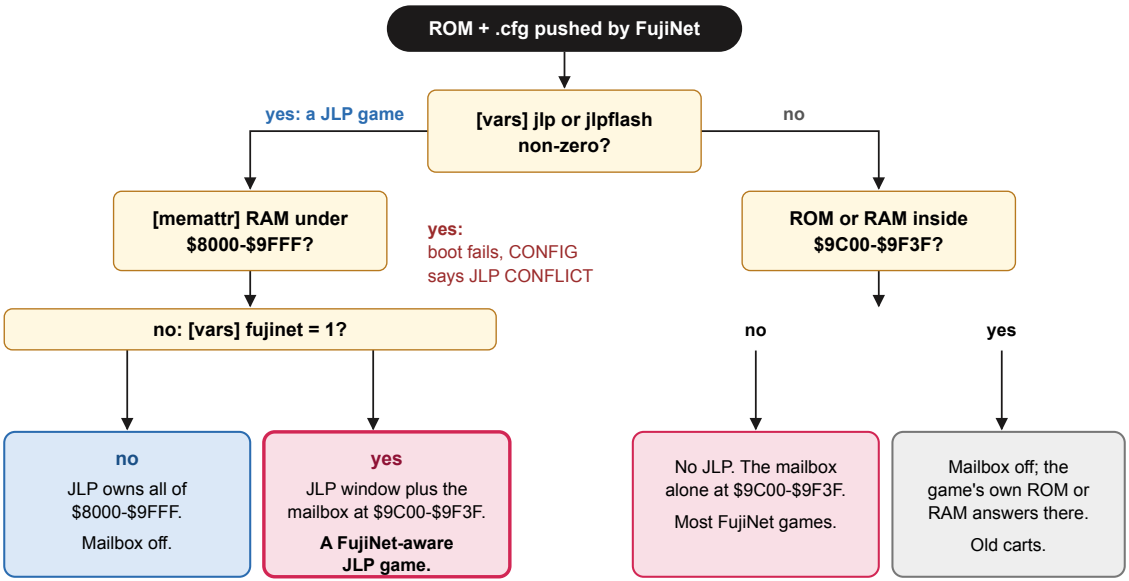
THE GAME'S .CFG	$8000-$9FFF	MAILBOX
no jlp or jlpflash	game ROM/RAM, if any	at `$9C00`, unless the game's own memory covers it
jlp, no fujinet	JLP	off
jlp and fujinet = 1	JLP	at `$9C00`–`$9F3F`
jlp and `[memattr]` RAM under `$8000`–`$9FFF`	—	boot fails: JLP CONFLICT

NOTE: A JLP game that knows nothing of FujiNet never sees a mailbox, so nothing it writes to high JLP RAM can start a network transaction. A game that does know asks for one with `fujinet = 1`.

WATCH OUT: Ship a `.bin` and `.cfg`, not a `.rom`. The cart ignores the metadata in an Intellicart `.rom` when booting over the network, so a `.rom` gets neither JLP nor the mailbox opt-in.

Who gets $8000-$9FFF at boot

fujinet-firmware pico/intellivision apply_boot_mapping(), from the .cfg pushed with the ROM



An Intellicart .rom carries no [vars] the cart reads: over the network it never gets JLP. Ship .bin + .cfg.

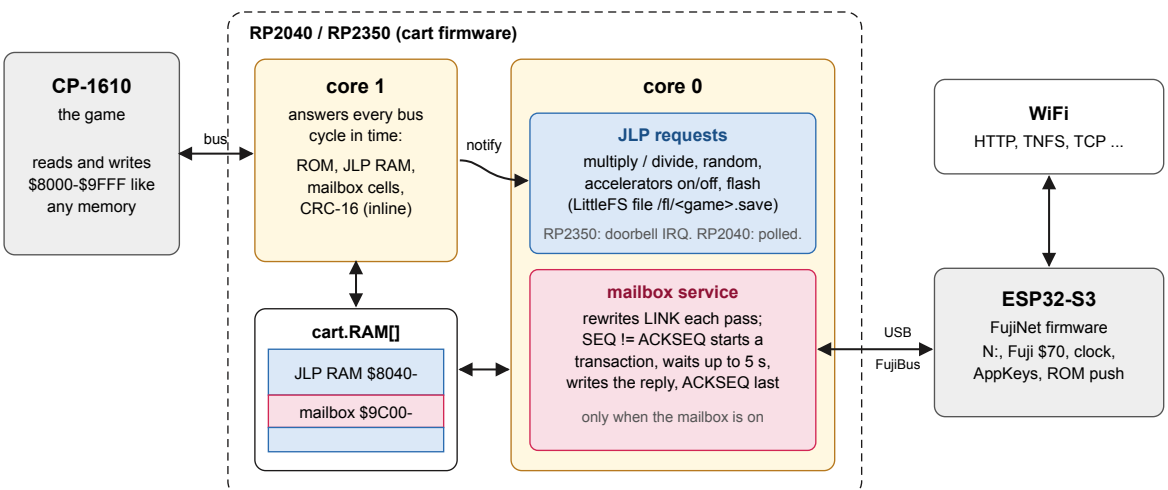
CHAPTER 3

INSIDE THE CARTRIDGE

Which processor answers which address, and what that means for timing.

Inside the cartridge

One RAM array holds JLP RAM and the mailbox. Two CPU cores split the work.



A flash command or a FujiNet transaction keeps core 0 busy; core 1 never stops serving the bus.

Core 1 of the RP2040 or RP2350 does nothing but answer the bus, in time, every cycle. It serves JLP RAM and the mailbox from the same array, and it runs the CRC-16 step itself. Everything slower goes to core 0: the multiply and divide results, the random-number buffer, flash commands (each one a file operation on the cart's own flash, in `/fl/<game>.save`), and the mailbox service that ships requests to the ESP32.

How core 0 hears about JLP work depends on the chip:

- **RP2350** (the FujiNet INTV board) rings a doorbell interrupt on every JLP register access, so the accelerators answer even while core 0 waits for the ESP32.
- **RP2040** (PiRTO II boards) checks for JLP work once per pass of its main loop. While it waits on a FujiNet transaction it doesn't check, so multiply and divide results and flash commands stall for as long as the transaction lasts.

A flash command keeps core 0 busy on both chips; the mailbox simply waits for it.

CHAPTER 4

WRITING A FUJINET-AWARE JLP GAME

Seven rules, all of them checkable at build time or at boot.

intybasic --jlp game.bas game.asm

Build with JLP support. 16-bit variables move into JLP RAM, starting at `$8040`.

FLASH INIT SIZE 6

If you use save-game flash, size it. IntyBASIC then writes `jlp = 3` and `jlpflash = 6` into the .cfg.

ASM CFGVAR "fujinet" = 1

Ask for the mailbox. as1600 copies this into `[vars]` beside `jlp` the cartridge and jzIntv both read it there.

as1600 -o game game.asm

No extension: as1600 writes `game.bin`, `game.cfg` and `game.rom`. Put the .bin and .cfg side by side on your FujiNet host.

NO MEMATTR

The FujiNet transport that the non-JLP games share starts with `ASM MEMATTR $8000, $9BFF, "+RWN"`. Leave it out: JLP already provides RAM there, and a `[memattr]` under `$8000`–`$9FFF` stops a JLP game from booting. The test cartridge's `src/fujinet.bas` is a transport without it.

STAY BELOW $9C00

IntyBASIC only warns when variables pass `$9F80`, so a large `DIM` can run straight across the mailbox. Look up `_SYSTEM2` in the .sym file; it must not exceed `$9C00`.

SEQ COMES FROM THE CART

```
#fn_seq = (PEEK($9C04) AND 255) + 1
IF #fn_seq > 255 THEN #fn_seq = 1
POKE $9C03, #fn_seq
```

A console reset restarts your program but not the cartridge, so derive each sequence number from the cart's `ACKSEQ`, skipping 0.

FLASH ROWS START AT FLASH.FIRST

The first row is 0 on the cartridge and 224 in jzIntv. Count from `FLASH.FIRST`, and never point a flash command's RAM address into the mailbox.

THINGS WORTH KNOWING

- **IntyBASIC 1.5** clears all of `$8040`–`$9F7F` at startup with `--jlp`, mailbox header included; 1.4.2 clears only its own variables. The mailbox still works, but its `F`, `N` magic is gone. The cart rewrites `LINK` (`$9C0F`) on every pass, so a `$A5` written there turning back into 0 or 1 proves a live mailbox.
- **Accelerators off** (`$8034`) leaves the mailbox working. On the cart, JLP RAM stays visible while they are off if the game has flash; real JLP and jzIntv hide it.
- **Multiplication is an accelerator access.** With `--jlp`, `*` by anything but a small constant, `/` and `MOD` become JLP register accesses. On an RP2040 cart that matters while a transaction is in flight.

CHAPTER 5

THE TEST CARTRIDGE

Twenty-four tests in four groups, run at power-up.

Boot `jlpfn.bin` from CONFIG. The tests run at once, ten to a screen:

```
JLP+FUJINET TEST 1/4
A1 JLP RAM  PASS   48
A2 MAILBOX  PASS    1
A3 VAR TOP  PASS 8DDE
A4 MBXSAFE  PASS 3618
A5 SLACK    PASS   64
A6 JLP OFF  PASS    2
B1 MARCH    PASS 3618
B2 MULTPLY  PASS   96
B3 DIVIDE   PASS   48
B4 CRC16    PASS    8
P23 F 0 S 0 I 1
```

```
STRESS   CLEAR=STOP

PASSES      39
FAILED       0
JLP OPS    998
JLP ERR      0
LAST         0
TXN FR       1
```

Each test shows PASS, FAIL, SKIP or INFO and one value. Press 1 to run them again, 2 for the stress loop, 3 or the disc to turn the page. The fourth page shows the SSID, IP address, firmware version, time, and flash and transaction timings.

TEST	WHAT IT PROVES
A1 JLP RAM	16-bit JLP RAM is present and flash has the 48 rows the .cfg asked for.
A2 MAILBOX	The mailbox answers inside the JLP window: magic, and a live `LINK`. If not, every FujiNet test skips.
A3 VAR TOP	IntyBASIC's variables end below `$9C00`.
A4 MBXSAFE	Writing every free JLP RAM word up to `$9BFF` leaves the mailbox header alone.
A5 SLACK	`$9F40`–`$9F7F` is full 16-bit JLP RAM.

TEST	WHAT IT PROVES
A6 JLP OFF	Accelerators off and on again: the mailbox stays, variables survive.
B1 MARCH	Four address-dependent patterns through all free JLP RAM.
B2 MULTPLY	All four multiplies against software, plus IntyBASIC's *.
B3 DIVIDE	Signed and unsigned divide and remainder, plus / and MOD.
B4 CRC16	Known answers computed on the host.
B5 RANDOM	Not stuck, not repeating every four reads, not zeros.
B6 FLASH	Erase, write and read back rows in two sectors, timed.
B7 ERASE	Erasing row 8 erases sector 1, not sector 0.
B8 RANGE	A flash read that would run past $9F7F is refused.
B9 BASICFL	The same through IntyBASIC's FLASH statements.
C1–C3	WiFi status, the adapter config, the clock device $45.
C4 APPKEY	64 fresh bytes written and read back (needs an SD card).
C5 HTTPGET	A 3,072-byte download into JLP RAM, every byte checked.
D1 ASYNC	The same download with JLP work running during every READ, every result checked.
D2 NET CRC	The JLP CRC unit checks the downloaded data against the host's CRC.
D3 NET>FL	The downloaded data to flash and back.
D4 PERSIST	At the next power-up, the previous run's flash copy is still there.

CHAPTER 6

EMULATORS AND KNOWN ISSUES

What jzIntv does now, and what the test cartridge found.

Since jzintv pull request 3, FujiNetWIFI's jzIntv models the cartridge, and the FujiNet Go Intellivision apps follow: the FujiNet peripheral answers `$9C00`–`$9F3F` only, a JLP game keeps the mailbox only with `fujinet = 1` in its .cfg (loaded from the command line or pushed by FujiNet), JLP stands aside on the mailbox, and `LINK` is refreshed continuously.

Earlier builds put the mailbox on top of JLP. jzIntv ANDs together every device that answers an address, so the handshake never completed and the multiply, divide, CRC and random registers read zero. On such a build the test cartridge finds no mailbox and fails A5, B2–B5 and B8.

ISSUE	FOUND BY	STATUS
Flash erase picks the sector as row % 8, not row / 8	B7	fix in progress
Flash row and RAM address are not range-checked	B8	fix in progress
RP2040: JLP stalls during a FujiNet transaction	D1, B9	fix in progress
jzIntv put the mailbox on top of JLP	A2, A5, B2–B5	fixed
`jlp = 2` means accelerators on to the cart, off to JLP	—	open
An Intellicart `.rom` booted over the network gets no JLP	—	open
The cart lets a flash row be written twice without erasing	B6	open
Without flash, `$8033` can't turn the accelerators back on	—	open
IntyBASIC 1.5 clears the mailbox at startup	A2	open

Source, ROM and the latest version of this booklet:
github.com/FujiNetWIFI/fujinet-intv-jlp-test